

Bilan de campagne de test

I. Recommandations

A. Tests à automatiser

Test 1 : Connexion utilisateur

La connexion utilisateur est un scénario essentiel à automatiser, car elle constitue la première étape pour accéder aux fonctionnalités principales comme le panier, les avis ou l'historique des commandes. En cas d'échec, l'utilisateur est immédiatement bloqué, ce qui peut entraîner une perte de clients. Ce test permet de vérifier le bon fonctionnement de l'authentification, la gestion des erreurs courantes et les redirections attendues. Son automatisation facilite la détection rapide des anomalies critiques et limite les régressions lors des évolutions de l'application.

Test 2 : Ajout au panier

L'ajout au panier est un test essentiel à automatiser, car il a un impact direct sur les ventes : si le panier ne fonctionne pas correctement, les utilisateurs ne peuvent pas finaliser leurs achats, ce qui entraîne une perte immédiate de revenus. Ce scénario couvre plusieurs cas critiques, comme l'ajout de produits, la gestion des stocks après ajout, les limites de quantités (produit indisponible, quantité négative ou trop élevée) ainsi que le passage en caisse, ce qui le rend long et répétitif à tester manuellement. Son automatisation permet donc de détecter rapidement les erreurs liées au stock ou à l'API, susceptibles d'impacter directement l'expérience utilisateur et les ventes.

La priorité est donnée à l'automatisation de la connexion et du panier, car ces fonctionnalités sont essentielles à l'accès au site, aux ventes et à l'expérience utilisateur.

B. Préconisations pour la suite

Dans une logique d'amélioration continue et lorsque le budget le permettra, il est recommandé d'étendre progressivement l'automatisation, notamment aux scénarios d'affichage des produits afin de vérifier la présence des informations essentielles comme les images, les prix et la disponibilité, en combinant des tests fonctionnels et des outils de vérification visuelle.

Cette démarche peut être complétée par la mise en place de tests réguliers sur différents navigateurs et appareils pour assurer la compatibilité, ainsi que par la priorisation de l'automatisation d'autres fonctionnalités critiques telles que le paiement et la gestion des commandes.

En parallèle, les tests très visuels ou fortement dépendants de l'esthétique et des animations, plus coûteux à maintenir et moins stables en E2E, peuvent être conservés en tests manuels.

L'intégration d'un processus automatisé exécutant les tests après chaque modification du code permettrait enfin de détecter rapidement les erreurs et de garantir la stabilité des fonctionnalités existantes, tout en équilibrant coût, maintenance et valeur ajoutée pour Eco Bliss Bath.

II. Tests automatisés

Document revu par :

Nom	Fonction	Version	Signature
Michael DSV	Testeur d'automatisation	1.0	MDSV

Contexte

Cette campagne de tests a été menée dans le cadre de la mise en production de la version 1.0 du site e-commerce **Eco Bliss Bath**, dédié à la commercialisation de produits de beauté respectueux de l'environnement.

À la suite d'une première phase de validation fonctionnelle effectuée manuellement par Marie, en charge des tests exploratoires, j'ai pris en charge la mise en œuvre d'une stratégie de tests automatisés. L'objectif était d'assurer la robustesse des fonctionnalités essentielles du site et de sécuriser le parcours utilisateur dès le lancement de la plateforme.

L'équipe mobilisée pour cette campagne est constituée de :

- Marie (test manuel)
- Michael DSV (test automatisé)
- Fabio (CTO, superviseur technique)

Les tests ont été réalisés avec l'outil Cypress, en environnement de développement local via Docker, sur le navigateur Edge, et sur le système Windows 11.

Objectifs

À l'approche du déploiement initial du site e-commerce **Eco Bliss Bath**, la sécurisation des fonctionnalités clés s'est imposée comme un enjeu majeur afin d'assurer un parcours utilisateur cohérent et de soutenir les objectifs commerciaux de la plateforme.

Pour répondre à ces exigences de qualité, l'équipe de développement a retenu **Cypress** comme solution de tests automatisés, permettant d'améliorer la fiabilité du produit tout en facilitant l'identification rapide des anomalies.

Encadrée par le CTO et en étroite collaboration avec la QA manuelle, j'ai eu pour mission de concevoir et d'exécuter une campagne de tests automatisés ciblée sur les parcours critiques.

L'objectif de cette démarche est de déployer une première série de tests automatisés visant à sécuriser les parcours fonctionnels jugés essentiels lors des tests manuels, à savoir **l'authentification des utilisateurs et la gestion du panier**.

Ces fonctionnalités, directement liées à l'accès aux services et à la finalisation des achats, représentent des points clés du parcours client et ont donc été priorisées dans la stratégie d'automatisation.

En parallèle, des tests orientés API ainsi que des Smoke tests ont été intégrés afin de vérifier la stabilité générale de l'application et de détecter rapidement toute anomalie critique lors des phases de déploiement.

Les objectifs secondaires de cette campagne sont :

- Évaluer la robustesse générale de l'application grâce à l'exécution de Smoke tests
- Valider le bon fonctionnement des principaux services exposés par l'API (authentification, gestion du panier, catalogue produits)
- Mettre en situation des cas atypiques ou limites via des tests UI afin d'identifier des comportements inattendus
- Centraliser les résultats obtenus, analyser les écarts observés et formaliser les anomalies dans un rapport clair et exploitable

Cette campagne de tests a également pour ambition de :

- Réduire les délais de détection des anomalies liées aux évolutions du code
- Sécuriser la qualité et la stabilité du site avant sa mise en production
- Initier une base solide pour une démarche d'automatisation des tests pérenne et évolutive

Tests effectués

Test 1 : API – authTests.cy.js

- **Requêtes POST de connexion :**
 - Connexion avec des identifiants valides : renvoie un code 200.

- Connexion avec des identifiants invalides : renvoie un code 401.

Test 2 : API – auth403OrdersTests.cy.js

- **Requête GET sur le panier (/orders) sans authentification :**
 - Sans token : l'accès au panier est refusé et renvoie un code **401 / 403** (non autorisé).

Test 3 : API – cartTests.cy.js

- **Requêtes GET sur le panier (/orders) :**
 - Sans connexion : renvoie un code **401 / 403** (non autorisé).
 - Avec connexion : renvoie le panier (liste des produits présents) avec un code **200**.
- **Requête PUT d'ajout au panier (/orders/add) :**
 - Ajout d'un produit **en stock** : renvoie un code **200** et le panier est mis à jour.
 - Ajout d'un produit **en rupture** : renvoie un code **200 au lieu d'un refus attendu (anomalie détectée)**.

Test 4 : API – ordersTests.cy.js

- **Requête POST de validation de commande (/orders) :**
 - Validation d'une commande contenant un produit **en stock** : renvoie un code **200** (comportement attendu).
 - Validation d'une commande contenant un produit **en rupture** : la commande est **validée quand même** (au lieu d'être refusée) (**anomalie détectée**).
- **Requête PUT de modification de quantité (/orders/{id}/change-quantity) :**
 - Mise à jour de la quantité d'un produit du panier : renvoie un code **200** et la quantité est modifiée.
- **Requête DELETE de suppression d'un produit (/orders/{id}/delete) :**

- Suppression d'un produit du panier : renvoie un code **200** et le produit est retiré du panier.

Test 5 : API – productTests.cy.js

- **Requêtes GET Produits :**
 - GET **/products** : récupère la liste des produits (code **200**).
 - GET **/products/random** : récupère des produits aléatoires (code **200**).
 - GET **/products/{id}** : récupère le détail d'un produit (code **200**) et vérifie la présence des informations attendues (ex : nom, description, image, stock/disponibilité, prix).

Test 6 : UI – loginTests.cy.js

- **Test de parcours de connexion utilisateur :**
 - Vérifie l'affichage du formulaire de connexion (champ email, champ mot de passe, bouton).
 - Simule la saisie des identifiants et la connexion.
 - Vérifie la redirection / état "connecté" (ex : présence du bouton **"Mon panier"** et/ou apparition de **"Déconnexion"**).

Test 7 : UI – cartTestsUi.cy.js

- **Ajout au panier :**
 - Le produit est ajouté au panier depuis sa fiche.
 - Le stock est mis à jour après l'ajout (stock décrémenté).
 - Le produit apparaît dans le panier, et la présence est confirmée également via l'API (contrôle croisé UI + API).
- **Limites de saisie :**
 - Quantité **négative** : l'ajout est **bloqué**.
 - Quantité à zéro : l'ajout est autorisé alors qu'un blocage devrait exister (**anomalie détectée**).
 - Quantité **supérieure à 20** : l'ajout est **autorisé** alors qu'un blocage devrait exister (**anomalie détectée**).
- **Nettoyage du panier (via l'API) :**

- Suppression des éléments du panier en fin de test pour éviter les effets de bord sur les scénarios suivants.

Test 8 : API – reviewsTests.cy.js

Ajout d'un avis

- **Scénario testé :**
Envoi d'une requête POST sur l'endpoint /reviews afin de vérifier le comportement de l'API lors de l'ajout d'un avis utilisateur, avec des données valides, des données incomplètes et une tentative d'injection de code JavaScript (XSS).
- **Résultat attendu :**
 - L'avis est ajouté lorsque tous les champs obligatoires sont correctement renseignés (code 200).
 - La requête est refusée lorsque des champs obligatoires sont manquants (code 400).
 - Toute tentative d'injection de script JavaScript est bloquée par l'API et retourne une erreur (code 4xx).
- **Résultat obtenu :**
 - L'avis valide est correctement ajouté (code 200).
 - Les avis incomplets sont refusés (code 400).
 - Une requête contenant un script JavaScript est acceptée et retourne un code 200.

Test 9 : Smoke Tests – loginSmoke.cy.js

- **Vérifications rapides (fonctionnalité critique "Connexion") :**
 - Présence des champs email et mot de passe.
 - Présence du bouton de validation / connexion.
 - Objectif : s'assurer que l'écran de connexion est accessible et exploitable après déploiement.

Test 10 : Smoke Tests – productSmoke.cy.js

- **Vérifications rapides (fonctionnalité critique "Produits") :**

- Présence de la liste de produits / page produits accessible.
- Présence d'un bouton "Ajouter au panier" sur une fiche produit (ou accès à une fiche produit).
- Présence de l'indication de disponibilité / stock.

Test 11 : Smoke Tests – xssAddToCartSmoke.cy.js

- **Contrôle de sécurité – tentative XSS sur la route d'ajout au panier :**
 - Envoie une charge contenant du contenu XSS sur l'ajout au panier.
 - Vérifie que l'application ne permet pas l'injection (objectif : détecter une faille XSS sur un flux critique).

Résultats de tests

Les tests automatisés réalisés avec Cypress, complétés par les tests manuels effectués par Marie, ont permis d'identifier **six anomalies** impactant la fiabilité fonctionnelle, la sécurité et la conformité de l'API du site **Eco Bliss Bath**.

Les anomalies détaillées ci-dessous sont issues des scénarios exécutés lors de la campagne de tests API, UI et Smoke tests.

Anomalie 1 – API – cartTests.cy.js

Ajout d'un produit en rupture de stock

- **Scénario testé :** Ajout d'un produit dont le stock est à zéro via l'API panier.
- **Résultat attendu :**
La requête doit être refusée avec un code d'erreur HTTP (400 ou équivalent), indiquant l'indisponibilité du produit.
- **Résultat obtenu :**
La requête est acceptée et retourne un code 200, ajoutant le produit au panier malgré l'absence de stock.

- **Impact :**
Risque de commandes impossibles à honorer, incohérences de stock et dégradation de l'expérience utilisateur.

Anomalie 2 – UI – ordersTests.cy.js

Validation de la commande d'un produit en rupture de stock

- **Scénario testé :** L'utilisateur est connecté à l'application et tente de valider une commande contenant un produit dont le stock est à zéro.
- **Résultat attendu :** La validation de la commande doit être bloquée. L'interface doit empêcher la finalisation de la commande et afficher un message d'erreur indiquant que le produit est en rupture de stock.
- **Résultat obtenu :** La commande est validée malgré la rupture de stock.
L'API retourne un code **200**, confirmant la commande sans contrôle de la disponibilité du produit.
- **Impact :** Risque de commandes impossibles à honorer, incohérence entre le stock réel et les commandes enregistrées, et dégradation importante de l'expérience utilisateur ainsi que de la fiabilité du système de commande.

Anomalie 3 – UI – cartUiTests.cy.js

Validation de la quantité saisie lors de l'ajout au panier

- **Scénario testé :**
Saisie d'une quantité **égale à 0** puis d'une quantité **supérieure à 20** lors de l'ajout d'un produit au panier.
- **Résultat attendu :**
La quantité doit être **comprise entre 1 et une valeur maximale définie** (ex. 20).
Toute valeur en dehors de cet intervalle (0 ou > 20) doit empêcher l'ajout du produit au panier.

- **Résultat obtenu :**
La quantité **égale à 0** est acceptée.
La quantité **supérieure à 20** est également acceptée.
Le produit est ajouté au panier sans restriction ni message de validation.
- **Impact :**
Non-respect des règles de gestion du panier et du stock, pouvant entraîner des commandes incohérentes, des erreurs de traitement et une dégradation de l'expérience utilisateur.

Anomalie 4 – API – reviewsTests.cy.js

Absence de protection contre les injections XSS lors de l'ajout d'un avis

- **Scénario testé :**
Soumission d'un avis contenant une charge XSS
(`<script>alert("XSS")</script>`) dans le champ commentaire via l'API.
- **Résultat attendu :**
L'API doit bloquer la requête et retourner une erreur HTTP (400 ou équivalent), empêchant l'enregistrement de contenu malveillant.
- **Résultat obtenu :**
L'API accepte la requête et retourne un code 200, enregistrant l'avis contenant le script JavaScript.
- **Impact :**
Cette faille représente un risque de sécurité majeur pouvant permettre l'exécution de scripts malveillants côté client.

Anomalie 5 – API – auth403OrdersTests.cy.js (anomalie issue des tests manuels de Marie)

Accès à des données confidentielles sans authentification

- **Scénario testé :** Accès aux données du panier sans être connecté.
- **Résultat attendu :**
Retour d'une erreur HTTP 403 (accès interdit).
- **Résultat obtenu :**
Retour d'une erreur HTTP 401.

- **Impact :**
Mauvaise gestion des codes d'erreur HTTP, pouvant entraîner une mauvaise interprétation côté front-end et une gestion incorrecte des accès.

Anomalie 6 – API – cartTests.cy.js (*anomalie issue des tests manuels de Marie*)

Non-conformité du verbe HTTP pour l'ajout au panier

- **Scénario testé :** Ajout d'un produit disponible au panier via l'API.
- **Résultat attendu :**
Utilisation du verbe HTTP **POST**, conformément aux bonnes pratiques REST.
- **Résultat obtenu :**
L'API utilise le verbe **PUT** pour l'ajout d'un produit.
- **Impact :**
Non-respect des conventions REST, pouvant compliquer la maintenance, la lisibilité de l'API et son intégration côté front-end.

Synthèse

L'ensemble des anomalies détectées met en évidence des **points de vigilance majeurs** concernant :

- la gestion du stock,
- la validation des entrées utilisateur,
- la sécurité applicative,
- et la conformité des endpoints API.

Des **captures d'écran et des vidéos** ont été générées automatiquement par Cypress afin de faciliter l'analyse des anomalies et leur communication à l'équipe de développement.

Rapport d'incidents :

1. Ajout de produit en rupture de stock

1. Environnement

- Outil utilisé : Cypress
- Type de test : Test API automatisé
- Fichier de test : apiTests/cartTests.cy.js

2. Étapes pour reproduire le bug

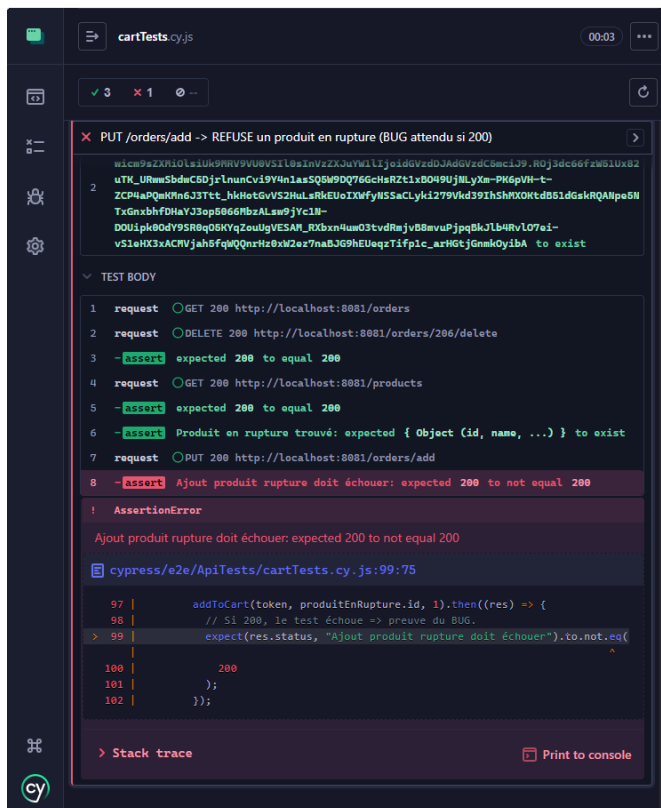
- Se connecter à l'application avec un utilisateur valide
- Identifier un produit dont le stock est à zéro
- Envoyer une requête d'ajout au panier via l'API

3. Comportement attendu

- Le produit ne doit pas être ajouté au panier
- L'API doit refuser la requête et retourner une erreur (code 400 ou équivalent)

4. Comportement observé

- Le produit est ajouté au panier malgré un stock à zéro
- L'API retourne un code 200 indiquant un succès



5. Message d'erreur

- Aucun message d'erreur bloquant n'est retourné par l'API

6. Autres informations

- L'absence de contrôle de disponibilité côté back-end permet de créer des commandes impossibles à honorer.

Priorité : Haute

Impact direct sur la gestion des stocks et la fiabilité des commandes.

2. Validation de commande avec un produit en rupture de stock

1. Environnement

- Outil utilisé : Cypress
- Type de test : Test API automatisé
- Fichier de test : apiTests/ordersTests.cy.js

2. Étapes pour reproduire le bug

- Ajouter un produit en rupture de stock au panier
- Tenter de valider la commande via l'endpoint /orders

3. Comportement attendu

- La validation de la commande doit être bloquée
- L'API doit retourner une erreur indiquant que la commande n'est pas valide

4. Comportement observé

- La commande est validée avec succès
- L'API retourne un code 200 malgré la présence d'un produit indisponible

```
ordersTests.cy.js 00:02 ...  
✓ 1 ✗ 1  
✗ POST /orders - échoue avec un produit EN RUPTURE (BUG attendu)  
2 e5j8nHalQjAcVzYx3uPRfpaQpJaxbsiR7hpyZrcen6PrFc6D4LR37HsaLG8q6ZFkc8-  
00GzlggEA_KkLUFL8GPzHmW12uYJ2EEyiyHumMXdC8b_4jveCGG-  
F_84oz9LHGHTZDGX2obv85iowaut6YmhzdTESPPvkCBKzVj3fT9Inum87VLAREuRd1VY6C398GquSt4QAtU  
UC1KKR7UpCVUVRQzalZ4MngiD3PMd3u8q4MCzPvtvP_cU4KCT8X1v27dZCoaoYkIgLLNEkdhkG6Us-aZga-  
TQATAdQxMfw1TRFwLW6tcH2S2M1Y2ePRow to exist  
TEST BODY  
1 request GET 404 http://localhost:8081/orders  
2 request GET 200 http://localhost:8081/products  
3 -assert expected 200 to equal 200  
4 -assert Produit en rupture trouvé: expected { Object (id, name, ...) } to exist  
5 request PUT 200 http://localhost:8081/orders/add  
6 writeFile cypress/logs/order_add_out_of_stock.json  
7 request POST 200 http://localhost:8081/orders  
8 -assert Validation commande doit échouer (rupture): expected 200 to not equal 200  
! AssertionError  
Validation commande doit échouer (rupture): expected 200 to not equal 200  
cypress/e2e/ApiTests/ordersTests.cy.js:116:22  
114 |         orderRes.status,  
115 |         "Validation commande doit échouer (rupture)"  
> 116 |     ).to.not.eq(200);  
    |           ^  
117 |     });  
118 |   });  
119 | }  
> Stack trace  
Print to console
```

5. Message d'erreur

- Aucun message bloquant n'est retourné

6. Autres informations

- Ce comportement peut générer des commandes fantômes et nuire à la crédibilité du site.

Priorité : Haute

3. Quantité invalide autorisée dans le panier (0 et > 20)

1. Environnement

- **Outil utilisé :** Cypress
- **Type de test :** Test UI automatisé
- **Fichier de test :** uiTests/cartTestsUi.cy.js

2. Étapes pour reproduire le bug

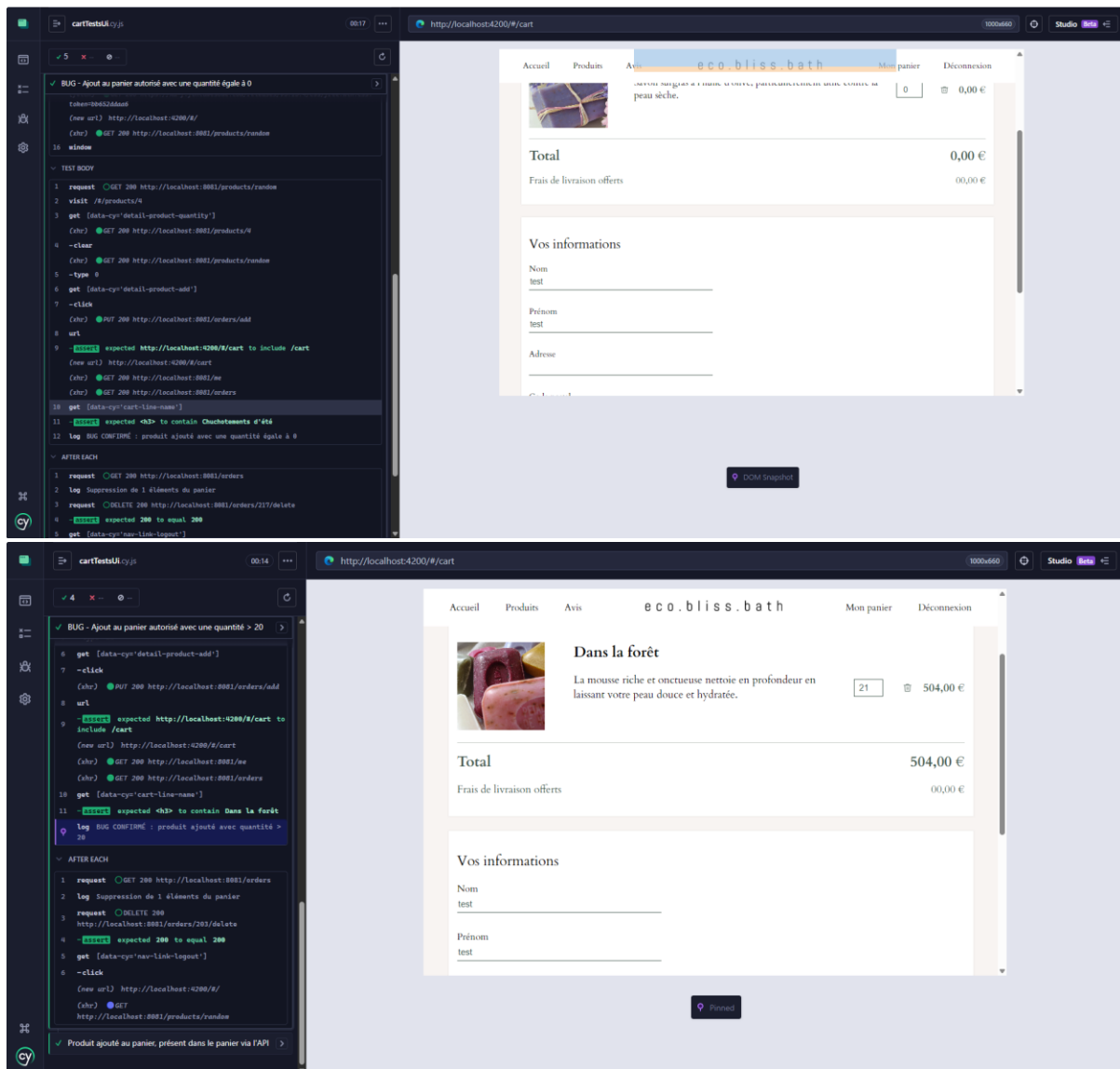
- Se rendre sur la fiche d'un produit
- Saisir une quantité égale à **0**
- Tenter d'ajouter le produit au panier
- Répéter l'opération en saisissant une quantité **supérieure à 20**
- Tenter à nouveau l'ajout au panier

3. Comportement attendu

- La quantité saisie doit être **strictement supérieure ou égale à 1**
- La quantité doit être **limitée à une valeur maximale définie** (ex. 20)
- Toute valeur en dehors de cet intervalle (**0 ou > 20**) doit bloquer l'ajout au panier via une règle de validation

4. Comportement observé

- Une quantité égale à **0** est acceptée
- Une quantité **supérieure à 20** est également acceptée
- Le produit est ajouté au panier **sans restriction ni message d'erreur**



5. Message d'erreur

- Aucun message d'erreur ou de blocage n'est affiché à l'utilisateur

6. Autres informations

- Ce comportement ne respecte pas les règles métier attendues pour la gestion des quantités
- Il peut entraîner des incohérences dans le panier, des erreurs de stock et des commandes invalides
- L'absence de validation côté front-end et/ou back-end augmente le risque d'anomalies en production

Priorité : Haute

4. Faille de validation XSS lors de l'ajout d'un avis

1. Environnement

- **Outil utilisé :** Cypress
- **Type de test :** Test API automatisé
- **Fichier de test :** apiTests/reviewsTests.cy.js

2. Étapes pour reproduire le bug

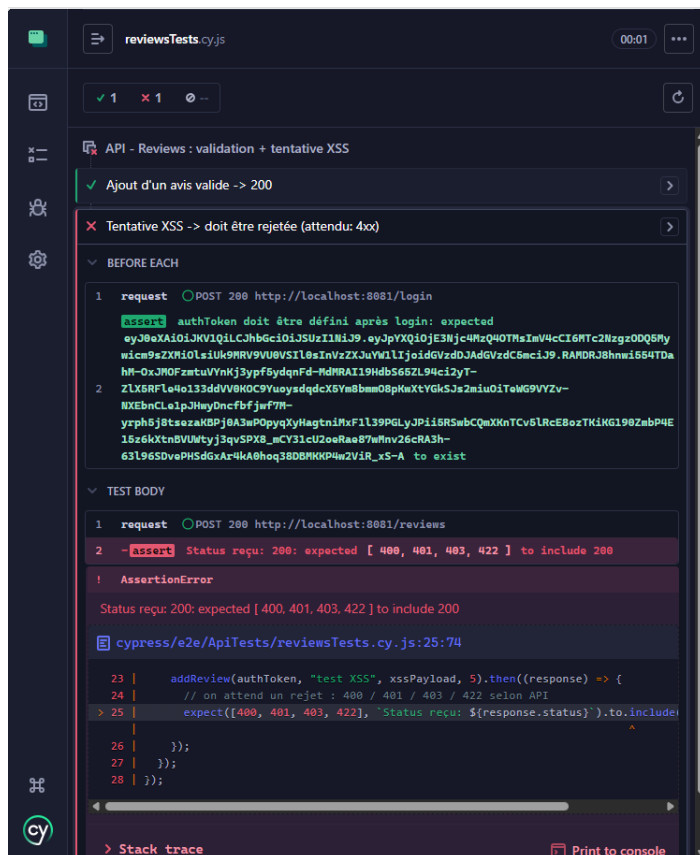
- Se connecter avec un utilisateur valide afin d'obtenir un token d'authentification
- Préparer une requête d'ajout d'avis via l'API /reviews
- Injecter un script JavaScript (payload XSS) dans le champ commentaire
- Soumettre la requête d'ajout d'avis

3. Comportement attendu

- L'API doit **rejeter la requête** contenant un contenu malveillant
- Un **code d'erreur HTTP (400 ou équivalent)** doit être retourné
- L'avis ne doit **pas être enregistré** côté serveur

4. Comportement observé

- La requête contenant le payload XSS est **acceptée**
- L'API retourne un **code HTTP 200**
- L'avis est traité comme valide malgré la présence de code malveillant



5. Message d'erreur

- Aucun message d'erreur bloquant n'est retourné par l'API

6. Autres informations

- L'absence de filtrage ou de validation du contenu utilisateur expose l'application à une **faille XSS**
- Ce type de vulnérabilité peut permettre :
 - L'exécution de scripts malveillants
 - Le vol de données utilisateur
 - Une atteinte à l'intégrité et à la confiance dans le site

Priorité : Très haute

5. Mauvais code HTTP pour l'accès à des données protégées (anomalie relevée par Marie)

1. Environnement

- Outil utilisé : Cypress
- Type de test : Test API automatisé
- Fichier de test : apiTests/auth403OrdersTests.cy.js

2. Étapes pour reproduire le bug

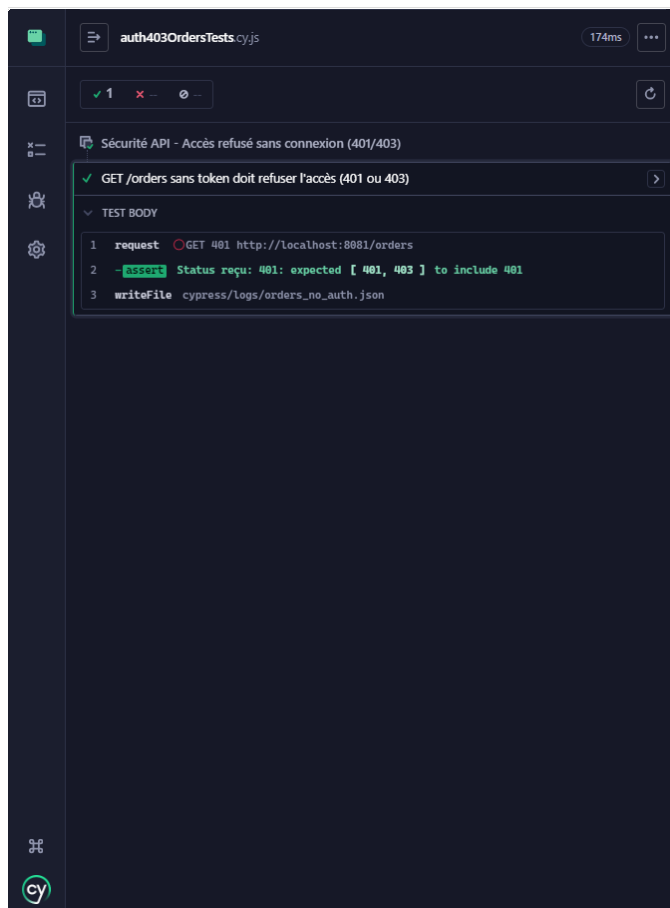
- Envoyer une requête vers une ressource protégée sans être authentifié

3. Comportement attendu

- L'API doit retourner une erreur 403 (accès interdit)

4. Comportement observé

- L'API retourne une erreur 401 à la place du code 403 attendu



5. Message d'erreur

- Message d'erreur incohérent avec le statut HTTP retourné

6. Autres informations

- Cette incohérence peut compliquer l'interprétation des erreurs côté front-end.

Priorité : Moyenne

6. Mauvais verbe HTTP pour l'ajout d'un produit au panier (Swagger)

1. Environnement

- Outil utilisé : Swagger
- Méthode testée : /orders/add

2. Étapes pour reproduire le bug

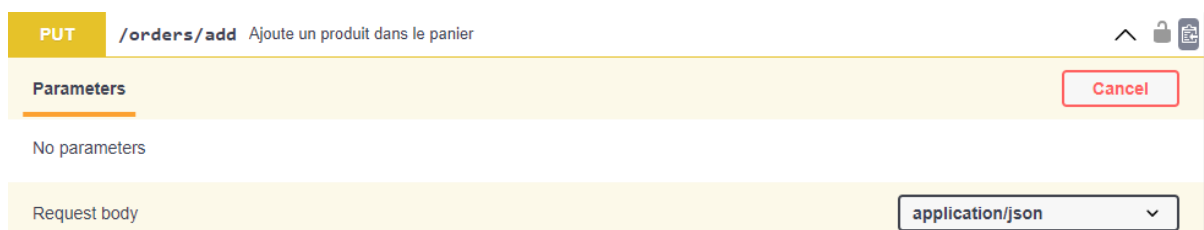
- Consulter la documentation Swagger
- Observer le verbe HTTP utilisé pour l'ajout d'un produit au panier

3. Comportement attendu

- L'ajout d'un produit doit utiliser la méthode POST (création de ressource)

4. Comportement observé

- La méthode PUT est utilisée pour l'ajout d'un produit



5. Message d'erreur

- Aucun message bloquant

6. Autres informations

- Cette implémentation ne bloque pas le fonctionnement mais ne respecte pas les conventions REST.

Priorité : Moyenne

7. Route de suppression redondante (Swagger)

1. Environnement

- Outil utilisé : Swagger
- Méthode testée : DELETE /orders/{id}/delete

2. Étapes pour reproduire le bug

- Consulter la documentation de l'API
- Observer la route de suppression d'un élément du panier

3. Comportement attendu

- Une route DELETE /orders/{id} est suffisante et conforme aux standards REST

4. Comportement observé

- La route inclut un suffixe /delete, redondant avec le verbe HTTP

The screenshot shows the Swagger UI for the endpoint `DELETE /orders/{id}/delete` with the description "Supprime un produit du panier". The "Parameters" tab is active, showing a table with one parameter:

Name	Description
<code>id</code> * required integer (path)	ID de la ligne du panier à supprimer

Below the table, there is a text input field with the value "id". A "Try it out" button is located in the top right corner of the parameters section.

5. Message d'erreur

- Aucun message bloquant

6. Autres informations

- Cette redondance nuit à la lisibilité et à la maintenabilité de l'API.

Priorité : Basse

Confiance

À l'issue de cette campagne de tests automatisés, la version actuelle de l'application **ne peut pas être considérée comme suffisamment fiable pour une mise en production immédiate**, sans corrections préalables.

Les tests réalisés ont permis de mettre en évidence plusieurs anomalies, dont certaines concernent des **fonctionnalités critiques du parcours utilisateur**, en particulier la gestion du panier, la validation des commandes et la sécurité des données saisies par les utilisateurs. Ces éléments ont un impact direct à la fois sur l'expérience utilisateur, la fiabilité métier et la sécurité globale de l'application.

Criticité des anomalies détectées

Anomalies de haute priorité (à corriger en urgence)

- **Ajout et validation de commandes avec des produits en rupture de stock**

L'application permet actuellement d'ajouter au panier et de valider une commande contenant des produits indisponibles.

Cette anomalie peut générer des commandes impossibles à honorer, entraînant des insatisfactions clients et une perte de crédibilité pour la plateforme.

- **Absence de contrôle sur les quantités saisies dans le panier**

Les tests UI ont montré que des quantités à 0 et excessives (supérieures à 20) sont acceptées sans aucun blocage.

Ce comportement peut provoquer des incohérences de stock et des erreurs côté serveur.

Anomalies de priorité moyenne

- **Utilisation d'un verbe HTTP inadapté pour l'ajout d'un produit au panier**

L'ajout d'un produit utilise la méthode PUT au lieu de POST, ce qui ne respecte pas les conventions REST.

Bien que cela ne bloque pas le fonctionnement, cette incohérence peut compliquer l'intégration front-end et la maintenance de l'API.

- **Route de suppression redondante dans l'API**

La présence d'une route DELETE /orders/{id}/delete est redondante avec le verbe HTTP utilisé.

Cette implémentation nuit à la lisibilité et à la cohérence globale de l'API.

Avis sur la stabilité de l'application

Points rassurants

- Le mécanisme de **connexion utilisateur fonctionne correctement**, avec une gestion cohérente des identifiants valides et invalides.
- Les **produits sont correctement affichés** et la navigation entre les pages principales ne génère pas d'erreurs bloquantes.
- Les **smoke tests** permettent de valider rapidement la disponibilité des fonctionnalités essentielles à chaque exécution, renforçant la détection précoce des régressions.

Points bloquants

- Les anomalies liées à la **gestion du stock et du panier et la faille sur les avis** remettent en question la fiabilité du processus de commande.

Conclusion

En l'état actuel, la version testée **ne permet pas une mise en production en toute confiance.**

Il est fortement recommandé de **corriger en priorité les anomalies critiques liées au panier, à la validation des commandes ainsi que la sécurité des avis** avant d'envisager un déploiement.

Une fois ces correctifs appliqués, une nouvelle campagne de tests automatisés devra être menée afin de confirmer la stabilité et la fiabilité de l'application.